

Foundation of Blockchain Technology (IS501)

Pre-Reading Material – Lecture 15

Function, Gas & Error Handling

Estimated Reading Time: 15–20 minutes

Before the Lecture

Functions in Solidity can perform different tasks. Some functions only **read** blockchain data, while others **change** it.

The view modifier is used when a function only reads blockchain state, while pure is used when a function neither reads nor changes blockchain state. Functions that change state require gas.

Understanding Gas

Gas is used to measure the computational work involved in executing operations on the blockchain.

Three important terms are:

- **Gas Price** – price paid per unit of gas
- **Gas Limit** – maximum gas allowed for a transaction
- **Gas Used** – gas actually consumed

A simple relationship is:

$$\text{Transaction Fee} = \text{Gas Used} \times \text{Gas Price}$$

Therefore, efficient Solidity code should avoid unnecessary computations and excessive storage operations.

Controlling Program Flow

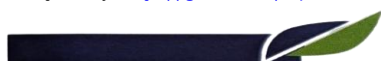
Like other programming languages, Solidity supports:

- if–else for decisions

Prepared by: Dr. Rajesh Kumar

Foundation of Blockchain Technology (IS501) | AY 2026–27

Course Repository: <https://github.com/rajesh101191/Foundation-of-Blockchain-Technology>



- while for repetition
- do-while for at-least-once execution
- for for repeated, structured operations

Since loops consume computational resources, they should be used carefully in smart contracts.

Handling Errors

Solidity provides three commonly used mechanisms:

- **require()** – checks conditions before continuing
- **revert()** – stops execution and rolls back changes
- **assert()** – checks conditions that should never fail

A simple way to remember them is:

require → **Prevent**

revert → **Roll back**

assert → **Check internal logic**

Think Before the Lecture

1. What is the difference between view and pure?
2. Why does changing blockchain data require gas?
3. What is the purpose of if-else?
4. When might require() be useful?
5. Why should smart contracts avoid unnecessary loops?

